

빅데이터 분석 연습문제 풀이

② Q1. 다음 단어 집합을 **Bag-of-Words** 방식으로 벡터화하시오. (대소문자 무시)

['Grape', 'orange', 'melon', 'Orange', 'Peach', 'grape']

③ 상세 풀이 >

Bag-of-Words (BoW)란?

Bag-of-Words는 텍스트를 수치 벡터로 표현하는 기법 중 하나입니다. 이름에서 알 수 있듯이, 텍스트에 포함된 단어들을 순서나 문맥에 상관없이 '가방'에 담겨있는 것으로 간주합니다. 이 방식은 오직 각 단어의 등장 빈도(frequency)에만 초점을 맞춰 텍스트를 표현합니다.

벡터화 과정은 다음과 같습니다.

1. 단어 집합(**Vocabulary**) 생성: 주어진 텍스트(여기서는 단어 리스트)에 있는 모든 고유한 단어를 모아 집합을 만듭니다. 문제의 조건에 따라 대소문자를 무시하므로, 모든 단어를 소문자로 변환하여 처리합니다.
2. 단어 빈도수 계산: 생성된 단어 집합의 각 단어가 원본 텍스트에서 몇 번 등장하는지 계산합니다.
3. 벡터 생성: 단어 집합을 기준으로 각 단어의 빈도수를 순서대로 나열하여 하나의 벡터로 만듭니다.

문제 풀이 적용:

1. 단어 소문자화 및 단어 집합 생성:
 - 주어진 단어 리스트: ['Grape', 'orange', 'melon', 'Orange', 'Peach', 'grape']
 - 모두 소문자로 변경: ['grape', 'orange', 'melon', 'orange', 'peach', 'grape']
 - 고유한 단어들을 추출하여 단어 집합 생성 (알파벳순 정렬): {grape, melon, orange, peach}
2. 단어 빈도수 계산:
 - grape : 2번
 - melon : 1번
 - orange : 2번
 - peach : 1번
3. 벡터 생성:

위에서 생성한 단어 집합 (grape, melon, orange, peach) 순서에 맞춰 각 단어의 빈도수를 나열합니다.

 - grape 의 빈도수: 2
 - melon 의 빈도수: 1
 - orange 의 빈도수: 2
 - peach 의 빈도수: 1

따라서 최종적인 Bag-of-Words 벡터는 다음과 같습니다.

[2, 1, 2, 1]

② Q2. 제조 회사는 전체 불량률 0.2%를 유지하고 있다. 품질 체크 부서를 기존 1개에서 5개로 늘린다. **Bonferroni Correction** 가정 시 각 부서의 허용 불량률은?

③ 상세 풀이 >

본페로니 교정(Bonferroni Correction)이란?

본페로니 교정은 다중 비교 문제(multiple comparisons problem)를 해결하기 위한 통계적 방법 중 하나입니다. 여러 가설을 동시에 검정할 때 전체 오류 확률(Family-wise error rate, FWER)이 증가하는 것을 막기 위해 사용됩니다.

예를 들어, 유의수준(p-value의 기준)을 0.05(5%)로 설정하고 가설 1개를 검정하면, 이 가설이 실제로 참인데도 우연히 기각될 확률은 5%입니다. 하지만 독립적인 가설 5개를 동시에 검정하면, 적어도 하나 이상의 가설이 우연히 잘못 기각될 확률은 $1 - (1 - 0.05)^5 \approx 0.226$ 으로 약 22.6%까지 치솟게 됩니다.

본페로니 교정은 이 문제를 매우 간단하고 보수적인 방법으로 해결합니다. 개별 검정에 사용할 유의수준을 원래 유의수준(α)에서 검정하는 가설의 개수(n)로 나누는 것입니다.

- 수정된 유의수준 (α') = α / n

이렇게 하면 모든 검정을 통틀어 적어도 하나의 오류를 범할 확률(FWER)이 원래 유의수준 α 이하로 유지되는 것이 보장됩니다.

문제 풀이:

주어진 정보:

- 전체 허용 불량률 (α): $0.2\% = 0.002$
- 품질 검사 부서 수(테스트 횟수, n): 1개에서 5개로 증가. 즉, $n = 5$

계산:

각 부서는 독립적으로 불량률을 검사한다고 볼 수 있습니다. 따라서 5개 부서가 각각 불량률을 검사하는 것은 5개 가설을 동시에 검정하는 것과 같습니다. 이 상황에서 전체 허용 불량률 0.2%를 유지하려면 본페로니 교정을 적용해야 합니다.

각 부서에 적용할 새로운 허용 불량률(수정된 유의수준)은 다음과 같이 계산됩니다.

- 각 부서의 허용 불량률 (α') = 전체 허용 불량률 (α) / 부서 수 (n)
- $\alpha' = 0.002 / 5$
- $\alpha' = 0.0004$

백분율로 변환하면 **0.04%** 입니다.

답:

본페로니 교정을 적용했을 때 각 부서의 허용 불량률은 **0.04%**가 되어야 합니다.

② Q3. 정규화(Normalization)의 필요성을 1가지 이상 설명하시오.

② 상세 풀이 >

정규화(Normalization)의 필요성

정규화는 데이터 전처리 과정에서 매우 중요한 단계입니다. 데이터셋의 여러 변수(feature)들의 값 범위(scale)가 서로 크게 다를 때, 이를 일정한 범위로 조정하는 역할을 합니다. 정규화가 필요한 주된 이유는 다음과 같습니다.

1. 머신러닝 모델 성능 향상

- 거리 기반 알고리즘: K-최근접 이웃(KNN), 서포트 벡터 머신(SVM), 클러스터링과 같이 데이터 간의 거리를 기반으로 작동하는 알고리즘은 값의 범위에 매우 민감합니다. 예를 들어, 한 변수는 0~1 사이의 값을 갖고 다른 변수는 0~1,000,000 사이의 값을 갖는다면, 거리 계산은 거의 두 번째 변수에 의해서만 결정될 것입니다. 이는 모델이 다른 변수의 중요도를 제대로 학습하지 못하게 만듭니다. 정규화를 통해 모든 변수가 모델 학습에 동등하게 기여하도록 만들 수 있습니다.
- 경사 하강법(Gradient Descent) 기반 알고리즘: 선형 회귀, 로지스틱 회귀, 신경망 등 경사 하강법을 사용하여 최적의 파라미터를 찾는 알고리즘에서도 정규화는 중요합니다. 변수들의 값 범위가 크게 다르면, 비용 함수(cost function)의 등고선이 한쪽으로 길게 찌그러진 타원 형태가 됩니다. 이 경우 최적점을 찾아가는 과정이 매우 비효율적이고 불안정

해질 수 있습니다. 정규화를 통해 등고선을 원형에 가깝게 만들어주면, 경사 하강법이 더 빠르고 안정적으로 최적점에 수렴할 수 있습니다.

2. 변수 간의 상대적 중요도 왜곡 방지

값의 범위가 큰 변수가 값의 범위가 작은 변수보다 더 중요하다고 잘못 해석될 여지를 없애줍니다. 정규화를 통해 모든 변수를 동일한 스케일에서 비교할 수 있습니다.

대표적인 정규화 방법:

- **최소-최대 정규화 (Min-Max Scaling):** 모든 값을 0과 1 사이의 값으로 변환합니다. $(X - X_{\min}) / (X_{\max} - X_{\min})$
- **표준화 (Standardization / Z-score Normalization):** 데이터의 분포를 평균이 0, 표준편차가 1인 표준 정규분포로 변환합니다. $(X - \text{mean}) / \text{std_dev}$

🔗 Q4. 버스 대기시간 데이터 `wait_time=[2,5,12,18,18,25,33,35,1,7]`

1. PMF 에서 $P(X = 18)$ 과 $P(X = 7)$ 을 계산하시오.
2. CDF 에서 $P(X \leq 20)$ 을 구하시오.

🔗 상세 풀이 >

주어진 데이터:

```
wait_time = [2, 5, 12, 18, 18, 25, 33, 35, 1, 7]
```

데이터 총 개수(n) = 10

데이터를 오름차순으로 정렬하면 다음과 같습니다.

```
[1, 2, 5, 7, 12, 18, 18, 25, 33, 35]
```

1) PMF(확률 질량 함수)에서 $P(X = 18)$ 과 $P(X = 7)$ 을 계산하시오.

PMF(Probability Mass Function, 확률 질량 함수)란?

PMF는 이산 확률 변수(discrete random variable)가 특정 값을 가질 확률을 나타내는 함수입니다. 어떤 변수 X 가 값 x 를 가질 확률을 $P(X = x)$ 로 표현합니다. 주어진 데이터 샘플에서 특정 값에 대한 PMF는 다음과 같이 계산할 수 있습니다.

- $P(X = x) = (x \text{가 나타난 횟수}) / (\text{전체 데이터 개수})$

문제 풀이:

- **$P(X = 18)$ 계산:**
 - `wait_time` 데이터에서 값 18은 **2번** 나타납니다.
 - 전체 데이터 개수는 **10개**입니다.
 - 따라서, $P(X = 18) = 2 / 10 = 0.2$
- **$P(X = 7)$ 계산:**
 - `wait_time` 데이터에서 값 7은 **1번** 나타납니다.
 - 전체 데이터 개수는 **10개**입니다.
 - 따라서, $P(X = 7) = 1 / 10 = 0.1$

2) CDF(누적 분포 함수)에서 $P(X \leq 20)$ 을 구하시오.

CDF(Cumulative Distribution Function, 누적 분포 함수)란?

CDF는 확률 변수 X 가 특정 값 x 보다 작거나 같을 확률을 나타내는 함수입니다. $P(X \leq x)$ 로 표현합니다. 즉, 가장 작은 값부터 x 까지의 모든 확률(PMF)을 더한 값입니다.

- $P(X \leq x) = (\text{값이 } x \text{보다 작거나 같은 데이터의 개수}) / (\text{전체 데이터 개수})$

문제 풀이:

• **P(X ≤ 20) 계산:**

- 정렬된 데이터 [1, 2, 5, 7, 12, 18, 18, 25, 33, 35] 에서 20보다 작거나 같은 값들을 찾습니다.
- 해당 값들은 [1, 2, 5, 7, 12, 18, 18] 입니다.
- 이 값들의 개수는 **7개**입니다.
- 전체 데이터 개수는 **10개**입니다.
- 따라서, $P(X \leq 20) = 7 / 10 = 0.7$

② Q5. 다음 RDD 연산 중 Transformation 만 모두 고르시오

함수명	반환형
flatMap()	<code>RDD[T] → RDD[U]</code>
reduceByKey()	<code>RDD[(K,V)] → RDD[(K,V)]</code>
collect()	<code>RDD[T] → Array[T]</code>
take(5)	<code>RDD[T] → Array[T]</code>
distinct()	<code>RDD[T] → RDD[T]</code>

③ 상세 풀이 >

RDD 연산의 종류: 변환(Transformation) 대 액션(Action)

Apache Spark의 RDD(Resilient Distributed Dataset) 연산은 크게 두 가지로 나뉩니다.

1. 변환(Transformation):

- 기존 RDD를 기반으로 새로운 **RDD**를 생성하는 연산입니다.
- 예를 들어, 모든 요소에 2를 곱하거나 특정 조건을 만족하는 요소만 필터링하는 등의 작업을 수행합니다.
- 변환 연산은 "지연 실행(Lazy Evaluation)"됩니다. 즉, 코드가 호출되는 즉시 실행되는 것이 아니라 액션 연산이 호출될 때까지 실행 계획(DAG)에 기록만 됩니다.
- 반환 유형(Return Type)은 `RDD[...]` 형태입니다.

2. 액션(Action):

- RDD에 대한 계산을 수행하고 그 결과를 드라이버 프로그램으로 반환하거나 외부 저장소에 저장하는 연산입니다.
- 액션 연산이 호출되는 시점에 해당 RDD를 생성하는 데 필요한 모든 변환이 실제로 실행됩니다.
- 반환 유형은 RDD가 아닌 다른 값(예: `Array[T]`, `Long`, `Unit` 등)입니다.

문제 풀이:

문제에 주어진 RDD 연산 중에서 변환을 선택하는 것은 연산의 결과로 새로운 **RDD**가 반환되는 것을 찾는 것과 같습니다. 각 함수의 반환 유형을 기준으로 판단할 수 있습니다.

함수 이름	반환 유형	연산 종류	설명
flatMap()	<code>RDD[T] → RDD[U]</code>	변환	입력 RDD를 받아 새로운 RDD[U] 를 반환합니다.
reduceByKey()	<code>RDD[(K,V)] → RDD[(K,V)]</code>	변환	입력 RDD를 받아 새로운 RDD[(K,V)] 를 반환합니다.
collect()	<code>RDD[T] → Array[T]</code>	액션	RDD의 모든 요소를 드라이버의 배열(Array)로 반환합니다.
take(5)	<code>RDD[T] → Array[T]</code>	액션	RDD의 상위 5개 요소를 드라이버의 배열(Array)로 반환합니다.
distinct()	<code>RDD[T] → RDD[T]</code>	변환	입력 RDD에서 중복을 제거한 새로운 RDD[T] 를 반환합니다.

답:

따라서 변환에 해당하는 연산은 `flatMap()`, `reduceByKey()`, `distinct()` 입니다.

② Q6. 다음 데이터에서 결측정보를 확인하세요. 결측치가 **MCAR**, **MAR** 여부인지 확인하세요. **MNAR** 에 대한 가능성이 있다고 할때, 그에 대한 예시를 적으시오.

ID	나이	근무경력(년)	급여	거주지
1	28	2	3200	서울
2	39	-	4500	부산
3	-	10	-	대전
4	45	20	7000	-
5	30	5	3800	서울

② 상세 풀이 >

결측치(Missing Value)의 종류

데이터 분석에서 결측치는 피할 수 없는 문제입니다. 결측이 발생하는 메커니즘을 이해하는 것은 이를 적절히 처리하는 데 매우 중요합니다. 결측 메커니즘은 주로 세 가지로 분류됩니다.

1. MCAR(Missing Completely At Random, 완전 무작위 결측):

- 결측이 발생할 확률이 관측된 데이터나 관측되지 않은 데이터 모두와 아무런 관련이 없는 경우입니다. 즉, 말 그대로 '완전히 무작위로' 결측이 발생합니다.
- 예: 설문조사 응답자가 실수로 특정 문항을 빠뜨리고 제출한 경우.

2. MAR(Missing At Random, 무작위 결측):

- 결측이 발생할 확률이 관측된 다른 변수에는 의존하지만, 결측된 값 자체와는 관련이 없는 경우입니다.
- 예: 남성보다 여성이 자신의 몸무게를 더 자주 응답하지 않는 경향이 있다면, '몸무게' 변수의 결측은 '성별'이라는 관측된 변수와 관련이 있습니다. 하지만 동일한 성별 내에서는 몸무게 값 자체(즉, 몸무게가 많이 나가서 숨기는 것)와는 관련이 없다고 가정합니다.

3. MNAR(Missing Not At Random, 비무작위 결측):

- 결측이 발생할 확률이 결측된 값 자체와 관련이 있는 경우입니다. 가장 처리하기 어려운 유형입니다.
- 예: 소득이 매우 높은 사람들이나 매우 낮은 사람들이 자신의 소득을 공개하기를 꺼려 소득 정보가 누락되는 경우. '소득' 변수의 결측이 '소득' 값 자체와 직접적인 관련이 있습니다.

문제 풀이:

1. 결측 정보 확인:

- ID 2: 근무경력(년) 결측
- ID 3: 나이, 급여 결측
- ID 4: 거주지 결측

2. MCAR, MAR 여부 확인:

- MCAR 가능성:** 이 데이터가 시스템 오류나 단순한 입력 실수로 인해 누락되었다면 MCAR로 볼 수 있습니다. 예를 들어, ID 4의 거주지 가 시스템상 주소 DB 연동 실패로 누락되었다면 이는 다른 변수와 아무 관련 없는 MCAR일 수 있습니다.
- MAR 가능성:**
 - ID 2의 근무경력 결측이 나이 나 급여 와 관련이 있을 수 있습니다. 예를 들어, 특정 나이대(예: 30대 후반)의 사람들이 이직 등의 이유로 경력 정보를 잘 입력하지 않는 경향이 있다면 MAR로 볼 수 있습니다.

- ID 3의 나이, 급여 결측은 근무경력 과 관련이 있을 수 있습니다. 예를 들어, 경력이 10년인 사람들의 급여 정보가 특정 이유로 누락되는 경향이 있다면 이는 MAR입니다.

3. MNAR 가능성에 대한 예시:

MNAR은 결측된 값 자체가 원인이 되는 경우입니다. 다음과 같은 시나리오를 생각해볼 수 있습니다.

- ID 3의 급여 결측: 근무경력 이 10년차임에도 불구하고 급여 가 동년차의 다른 사람들에 비해 현저히 낮거나 높아서, 정보 제공을 꺼렸을 가능성이 있습니다. 이 경우, 급여가 '낮다' 또는 '높다'는 사실 자체가 결측의 원인이므로 MNAR에 해당합니다.
- ID 2의 근무경력 결측: 나이 가 39세인데 근무경력 이 매우 짧거나(예: 신입), 혹은 경력 단절 기간이 길어서 경력 정보를 입력하고 싶지 않았을 수 있습니다. 이 경우, '짧은 경력'이라는 값 자체가 결측의 원인이므로 MNAR입니다.

Q7. Spark 에서 하나의 Job 이 실행되는 과정에서 나타나는 Stage-Task 구조를 계층(Hierarchy) 관점에서 설명하시오.

상세 풀이 >

Spark 실행 계층 구조: Job, Stage, Task

Spark에서 하나의 코드를 실행하면, Spark는 이를 내부적으로 Job, Stage, Task라는 계층적인 단위로 나누어 분산 처리합니다. 이 구조는 Spark의 핵심인 DAG(Directed Acyclic Graph) 스케줄러에 의해 관리됩니다.

계층 구조:

```
[ Spark Application ]
  |
  [ Job ]
    |
    [ Stage 1 ]---[ Stage 2 ]---...
      |           |
      [ Task 1-1 ] [ Task 2-1 ]
      [ Task 1-2 ] [ Task 2-2 ]
      ...         ...
```

1. Application (애플리케이션):

- 가장 상위 개념으로, 사용자가 제출한 하나의 Spark 프로그램 전체를 의미합니다.
- 하나의 Spark Application은 하나 이상의 작업으로 구성됩니다.
- 예: spark-submit 으로 실행한 .py 또는 .jar 파일 하나.

2. Job (작업):

- Spark Application 내에서 하나의 액션 연산이 호출될 때마다 하나의 작업이 생성됩니다.
- 액션 연산은 RDD에 대한 실제 계산을 촉발하고 결과를 반환(예: count(), collect())하거나 저장(예: saveAsTextFile())하는 역할을 합니다.
- 하나의 작업은 여러 단계로 나뉘어 실행됩니다.

3. Stage (스테이지):

- 하나의 작업은 셔플이 발생하는 지점을 기준으로 여러 단계로 분리됩니다.
- 셔플이란 reduceByKey, groupByKey, join 과 같이 네트워크를 통해 데이터를 재분배하는 과정을 의미합니다. 즉, 스테이지는 셔플 없이 연속적으로 수행할 수 있는 변환들의 묶음입니다.
- 같은 스테이지 내의 태스크들은 데이터 이동 없이 파이프라이닝(pipelining) 방식으로 동시에 실행될 수 있어 매우 효율적입니다.

4. Task (태스크):

- 가장 작은 실행 단위로, 하나의 스테이지는 여러 태스크로 구성됩니다.
- 태스크는 하나의 파티션(Partition)에 대해 하나의 CPU 코어(Executor의 core)에서 실행되는 실제 작업 단위입니다.
- 예를 들어, 어떤 스테이지가 100개의 파티션을 가진 RDD를 처리한다면, 해당 스테이지는 100개의 태스크를 생성하여 각 파티션에 대해 동일한 연산을 병렬로 수행합니다.

요약:

사용자가 액션을 호출하면(애플리케이션), 하나의 작업이 생성됩니다. 이 작업의 실행 계획(DAG)을 분석하여 셔플을 기준으로 여러 스테이지로 나눕니다. 각 스테이지는 데이터 파티션의 개수만큼 태스크를 생성하여 Executor에서 병렬로 실행하는 구조입니다.

Q8. 다음 Spark 코드의 line 번호 중 cache 가 영향을 미치는 구간(연산 line)을 모두 쓰시오.

```
1 val data = sc.textFile("hdfs://...")
2 val filtered = data.filter(_.contains("INFO"))
3 val parsed = filtered.map(_.split(","))
4 parsed.cache()
5 parsed.filter(_(2).toInt > 50).count()
```

상세 풀이 >

Spark의 cache() 연산과 영향 범위

Spark에서 cache() 또는 persist() 는 RDD를 메모리나 디스크에 저장하여 재사용할 때의 성능을 향상시키는 매우 중요한 변환 연산입니다.

cache() 의 작동 원리:

- cache() 는 지연 실행(Lazy Evaluation)되는 변환입니다. 즉, parsed.cache() 줄이 호출되는 순간에는 아무 일도 일어나지 않습니다.
- cache() 가 적용된 RDD(parsed)에 대해 최초의 액션 연산(여기서는 count())이 실행될 때, Spark는 이 RDD를 계산하여 결과를 반환함과 동시에 RDD의 각 파티션을 Executor의 메모리(기본 설정)에 저장합니다.
- 이후에 동일한 RDD(parsed)를 사용하는 다른 액션 연산이 호출되면, Spark는 원본 데이터 소스(HDFS)로부터 복잡한 계산(textFile -> filter -> map)을 다시 수행하는 대신, 메모리에 캐시된 RDD를 즉시 읽어서 사용합니다. 이로 인해 연산 속도가 매우 빨라집니다.

문제 풀이:

주어진 Spark 코드는 다음과 같습니다.

```
1 val data = sc.textFile("hdfs://...")
2 val filtered = data.filter(_.contains("INFO"))
3 val parsed = filtered.map(_.split(","))
4 parsed.cache()
5 parsed.filter(_(2).toInt > 50).count()
```

cache() 가 영향을 미치는 구간(연산 줄):

cache() 는 parsed RDD와 그 이전의 모든 RDD 생성 과정에 영향을 미칩니다. 즉, parsed RDD를 계산하는 데 필요한 모든 연산을 "기억"하고 그 결과를 저장합니다.

- 5행(count() 액션)이 처음 실행될 때, parsed RDD가 없으므로 Spark는 DAG(실행 계획)에 따라 parsed 를 계산합니다.

2. `parsed` 를 계산하려면 부모 RDD인 `filtered` 가 필요하고, `filtered` 를 계산하려면 원본 RDD인 `data` 가 필요합니다.
3. 따라서 다음 연산이 순차적으로 실행됩니다.
 - 1행: `sc.textFile(...)` - HDFS에서 데이터를 읽어 `data` RDD를 생성합니다.
 - 2행: `data.filter(...)` - `data` RDD에서 "INFO"를 포함하는 행을 필터링하여 `filtered` RDD를 생성합니다.
 - 3행: `filtered.map(...)` - `filtered` RDD의 각 행을 `,` 를 기준으로 분리하여 `parsed` RDD를 생성합니다.
4. 4행의 `cache()` 명령에 따라 이렇게 생성된 `parsed` RDD의 내용이 각 **Executor**의 메모리에 저장됩니다.
5. 그런 다음 5행의 나머지 연산(`filter(...).count()`)이 수행됩니다.

만약 이 코드 뒤에 `parsed.filter(_.contains("ERROR")).count()` 와 같이 `parsed` RDD를 재사용하는 코드가 있는 경우, 1, 2, 3행을 다시 실행하지 않고 메모리에 저장된 `parsed` RDD를 즉시 사용합니다.

답:

`cache()` 의 영향을 받아 캐시를 생성하기 위해 실행되는 연산 라인은 **1, 2, 3**번입니다. `cache()` 는 이러한 연산의 결과인 `parsed` RDD를 저장하여 이 RDD가 재사용될 때 1, 2, 3번 라인이 다시 실행되는 것을 방지합니다.

㉠ **Q9. 4 일 동안의 지각 학생 수에 대한 관측값과 기대값이 다음과 같을 때, 유의수준 5%에서 귀무가설(“요일 간 차이가 없다”)의 기각 여부를 판단하시오.**

요일	관측값	기대값
월	5	7
화	10	7
수	8	7
목	5	7

(자유도 $df = 3$, 표 제공됨)

㉡ 상세 풀이 >

카이제곱 검정을 이용한 가설 검정

카이제곱 검정은 관측된 빈도가 기대 빈도와 통계적으로 유의미하게 다른지 판단하는 데 사용되는 가설 검정 방법입니다. 주로 범주형 데이터 분석에 사용됩니다.

가설 설정:

- **귀무가설(H_0):** 관측값과 기대값 사이에 차이가 없다. (즉, 요일별 지각 학생 수에 차이가 없다.)
- **대립가설(H_1):** 관측값과 기대값 사이에 차이가 있다. (즉, 요일별 지각 학생 수에 차이가 있다.)

카이제곱 통계량 계산:

카이제곱 통계량(χ^2)은 각 범주에 대해 $(\text{관측값} - \text{기대값})^2 / \text{기대값}$ 을 모두 더한 값입니다.

- $\chi^2 = \sum [(O - E)^2 / E]$
 - O: 관측 빈도
 - E: 기대 빈도

문제 풀이:

주어진 데이터:

요일	관측값 (O)	기대값 (E)	(O - E)	(O - E) ²	(O - E) ² / E
월	5	7	-2	4	4 / 7 ≈ 0.571
화	10	7	3	9	9 / 7 ≈ 1.286
수	8	7	1	1	1 / 7 ≈ 0.143
목	5	7	-2	4	4 / 7 ≈ 0.571
합계	28	28			$\chi^2 \approx 2.571$

1. 카이제곱 통계량 계산:

$$\begin{aligned}\chi^2 &= (5-7)^2/7 + (10-7)^2/7 + (8-7)^2/7 + (5-7)^2/7 \\ &= 4/7 + 9/7 + 1/7 + 4/7 \\ &= 18/7 \\ &\approx 2.571\end{aligned}$$

2. 기각 여부 판단:

계산된 카이제곱 통계량을 기각역(critical value)과 비교하여 귀무가설의 기각 여부를 판단합니다. 기각역은 유의수준(α)과 자유도(df)에 따라 결정됩니다.

- 유의수준 (α): 0.05 (5%)
- 자유도 (df): 범주의 수(k) - 1 = 4 - 1 = 3

문제에서 "표 제공됨"이라고 언급했으므로, 우리는 카이제곱 분포표에서 유의수준 0.05, 자유도 3에 해당하는 임계값을 찾아야 합니다.

일반적인 카이제곱 분포표에서, **df=3, $\alpha=0.05$** 일 때의 임계값은 **7.815**입니다.

3. 결론:

- 계산된 카이제곱 통계량($\chi^2 \approx 2.571$)은 임계값(7.815)보다 작습니다.
- **2.571 < 7.815**
- 통계량이 임계값보다 작으므로, 귀무가설을 기각할 충분한 근거가 없습니다.

답:

계산된 카이제곱 통계량(약 2.571)이 유의수준 5%, 자유도 3에서의 임계값 7.815보다 작으므로, 귀무가설("요일 간 차이가 없다")을 기각할 수 없습니다. 즉, 요일별 지각 학생 수의 차이는 통계적으로 유의미하다고 볼 수 없습니다.

② Q10. 다음은 임의의 3×3 행렬 A에 대한 SVD 결과이다. SVD의 특이값은 다음과 같다.

$$\sigma_1 = 7, \sigma_2 = 3, \sigma_3 = 1$$

- 차원 축소로, 상위 k=2 개만 취하는 경우, 전체 에너지 대비 오차의 비율(오차율, Error Ratio)을 계산하시오.

③ 상세 풀이 >

SVD(Singular Value Decomposition, 특이값 분해)와 에너지

SVD는 행렬을 세 개의 행렬(U, Σ , V^T)의 곱으로 분해하는 기법으로, 차원 축소, 노이즈 제거 등 다양한 분야에 활용됩니다. 특히 특이값(Singular Value)은 원본 행렬의 데이터가 각 특이 벡터 방향으로 얼마나 큰 분산을 갖는지, 즉 해당 방향의 '에너지' 또는 '중요도'를 나타내는 값으로 해석할 수 있습니다.

- 전체 에너지: 전체 에너지는 모든 특이값의 제곱의 합으로 정의됩니다.
- 차원 축소: SVD를 이용한 차원 축소는 중요한 정보(에너지)를 대부분 가진 상위 k개의 특이값과 그에 해당하는 특이 벡터들만 남기고 나머지는 버리는 방식으로 이루어집니다.
- 오차(Error): 차원 축소 과정에서 버려진 특이값들은 손실되는 정보를 의미하며, 이것이 오차를 발생시킵니다.

오차율(Error Ratio) 계산:

오차율은 전체 에너지 중에서 버려진 특이값들에 해당하는 에너지가 차지하는 비율로 계산됩니다.

- 전체 에너지(Total Energy) = $\Sigma (\sigma_i^2)$ (모든 특이값의 제곱의 합)
- 손실된 에너지(Lost Energy) = $\Sigma (\sigma_j^2)$ (버려진 특이값들의 제곱의 합)
- 오차율(Error Ratio) = 손실된 에너지 / 전체 에너지

문제 풀이:

주어진 정보:

- 전체 특이값: $\sigma_1 = 7, \sigma_2 = 3, \sigma_3 = 1$
- 차원 축소: 상위 $k=2$ 개만 취함.
 - 선택된 특이값: $\sigma_1 = 7, \sigma_2 = 3$
 - 버려진 특이값: $\sigma_3 = 1$

1. 전체 에너지 계산:

$$\text{전체 에너지} = \sigma_1^2 + \sigma_2^2 + \sigma_3^2$$

$$= 7^2 + 3^2 + 1^2$$

$$= 49 + 9 + 1$$

$$= 59$$

2. 손실된 에너지 계산:

손실된 에너지는 버려진 특이값(σ_3)의 제곱과 같습니다.

$$\text{손실된 에너지} = \sigma_3^2$$

$$= 1^2$$

$$= 1$$

3. 오차율 계산:

$$\text{오차율} = \text{손실된 에너지} / \text{전체 에너지}$$

$$= 1 / 59$$

답:

상위 2개 특이값만 사용하는 경우, 전체 에너지 대비 오차 비율(오차율)은 **1/59**입니다. (약 1.69%)

② Q11. Bloom Filter 정보가 다음과 같을 때, false positive 가 발생할 수 있는 원소를 찾으시오.

- bit array size = 12
- hash functions :
 - $h_1(x) = x \% 12$
 - $h_2(x) = (x // 10 + x \% 10) \% 12$
- inserted items : [15, 27, 39]
- test items : [11, 24, 36, 51]

② 상세 풀이 >

블룸 필터(Bloom Filter)와 거짓 양성(False Positive)

블룸 필터는 어떤 원소가 집합에 속하는지 여부를 **확률적으로** 검사하는 데이터 구조입니다. 매우 적은 메모리를 사용하여 빠른 검사가 가능하지만, 두 가지 특징이 있습니다.

- 거짓 양성(False Positive) 가능성: 집합에 없는 원소를 '있다'고 잘못 판단할 수 있습니다.
- 거짓 음성(False Negative) 없음: 집합에 있는 원소를 '없다'고 판단하는 경우는 절대 없습니다.

작동 원리:

1. 초기화: 크기가 m 인 비트 배열을 준비하고 모든 비트를 0으로 초기화합니다.
2. 추가(Insertion): 원소 x 를 추가할 때, k 개의 다른 해시 함수 $h_1, h_2, \dots, h_k$ 를 사용하여 x 를 해싱합니다. 각 해시 함수의 결과값(인덱스)에 해당하는 비트 배열의 위치를 1로 설정합니다.
3. 검사(Testing): 원소 y 가 집합에 있는지 검사할 때, k 개의 해시 함수를 똑같이 적용합니다. 각 해시 함수의 결과값에 해당하는 비트 배열의 위치가 모두 1이면 '있다'고 판단하고, 하나라도 0이면 '없다'고 판단합니다.

거짓 양성 발생 이유:

서로 다른 원소들을 추가하는 과정에서, 해시 충돌로 인해 우연히 특정 위치의 비트들이 모두 1로 채워질 수 있습니다. 이 경우, 한 번도 추가된 적 없는 원소임에도 불구하고 해당 원소를 검사했을 때 모든 비트가 1이 되어 '있다'고 잘못 판단하게 됩니다.

문제 풀이:

주어진 정보:

- bit array size = 12 (크기 12의 비트 배열, 인덱스 0~11)
- hash functions :
 - $h_1(x) = x \% 12$
 - $h_2(x) = (x // 10 + x \% 10) \% 12$
- inserted items : [15, 27, 39]
- test items : [11, 24, 36, 51]

1. inserted items 를 블룸 필터에 추가:

- 초기 비트 배열: [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
- 원소 15 추가:
 - $h_1(15) = 15 \% 12 = 3$
 - $h_2(15) = (15 // 10 + 15 \% 10) \% 12 = (1 + 5) \% 12 = 6$
 - 비트 배열의 3번, 6번 인덱스를 1로 설정.
 - 비트 배열: [0, 0, 0, 1, 0, 0, 1, 0, 0, 0, 0, 0]
- 원소 27 추가:
 - $h_1(27) = 27 \% 12 = 3$
 - $h_2(27) = (27 // 10 + 27 \% 10) \% 12 = (2 + 7) \% 12 = 9$
 - 비트 배열의 3번, 9번 인덱스를 1로 설정. (3번은 이미 1)
 - 비트 배열: [0, 0, 0, 1, 0, 0, 1, 0, 0, 1, 0, 0]
- 원소 39 추가:
 - $h_1(39) = 39 \% 12 = 3$
 - $h_2(39) = (39 // 10 + 39 \% 10) \% 12 = (3 + 9) \% 12 = 0$
 - 비트 배열의 3번, 0번 인덱스를 1로 설정. (3번은 이미 1)
 - 최종 비트 배열: [1, 0, 0, 1, 0, 0, 1, 0, 0, 1, 0, 0]

2. test items 를 검사하여 거짓 양성 찾기:

각 test item 에 대해 해시 함수를 적용하고, 최종 비트 배열의 해당 위치가 모두 1인지 확인합니다.

- 원소 11 검사:
 - $h_1(11) = 11 \% 12 = 11$
 - $h_2(11) = (1 + 1) \% 12 = 2$
 - 비트 배열[11] = 0, 비트 배열[2] = 0. → '없음' (정상)
- 원소 24 검사:
 - $h_1(24) = 24 \% 12 = 0$
 - $h_2(24) = (2 + 4) \% 12 = 6$
 - 비트 배열[0] = 1, 비트 배열[6] = 1. → '있음' (거짓 양성!)

- 원소 24는 `inserted items` 에 없지만, 검사 결과는 '있음'으로 나왔습니다. 이는 15, 27, 39를 추가하는 과정에서 0번과 6번 비트가 우연히 모두 1로 채워졌기 때문입니다.
- 원소 36 검사:
 - $h_1(36) = 36 \% 12 = 0$
 - $h_2(36) = (3 + 6) \% 12 = 9$
 - 비트 배열[0] = 1, 비트 배열[9] = 1. → '있음' (거짓 양성!)
 - 원소 36도 `inserted items` 에 없지만, 검사 결과는 '있음'으로 나왔습니다.
- 원소 51 검사:
 - $h_1(51) = 51 \% 12 = 3$
 - $h_2(51) = (5 + 1) \% 12 = 6$
 - 비트 배열[3] = 1, 비트 배열[6] = 1. → '있음' (거짓 양성!)
 - 원소 51도 `inserted items` 에 없지만, 검사 결과는 '있음'으로 나왔습니다.

답:

거짓 양성이 발생하는 원소는 **24, 36, 51** 입니다.

② Q12. 다음 데이터는 **Flajolet-Martin** 알고리즘에서 해시 결과이다. 가장 오른쪽 1이 처음 등장하는 위치를 기반으로 고유 원소 수를 추정하시오.

값	해시값 (8bit)
A	01010000
B	00010010
C	00100000
D	10000001

③ 상세 풀이 >

Flajolet-Martin 알고리즘을 이용한 고유 원소 수 추정

Flajolet-Martin 알고리즘은 스트리밍 데이터에서 고유한 원소의 개수(카디널리티)를 근사적으로 추정하는 데 사용되는 확률적 알고리즘입니다. 매우 적은 메모리를 사용하여 대용량 데이터의 고유 원소 수를 빠르게 추정할 수 있습니다.

알고리즘 원리:

1. **해싱**: 스트림의 각 원소를 비트 문자열로 변환하는 해시 함수 $h(x)$ 를 사용합니다. 이 해시 함수는 결과값이 비트 전체에 고르게 분포되어야 합니다.
2. **$R(x)$ 계산**: 각 해시값 $h(x)$ 에 대해, 가장 오른쪽(최하위 비트)부터 시작해서 처음으로 1이 나타나는 위치를 $R(x)$ 로 정의합니다. 예를 들어, 해시값이 `...1000` 이면 $R(x)=3$ 입니다. (0부터 시작하면)
3. **최댓값 유지**: 스트림을 읽으면서 계산된 모든 $R(x)$ 값들 중 최댓값 R_{max} 를 계속 유지합니다.
4. **추정**: 스트림의 모든 원소를 처리한 후, 고유 원소의 개수를 $2^{R_{max}}$ 로 추정합니다.

직관적인 이해:

고유한 원소가 N 개 있다면, $R(x)=k$ 인 해시값을 가질 확률은 $1/2^{(k+1)}$ 입니다. 즉, $R(x)$ 가 크려면 해시값 뒤에 0이 많이 붙어야 하는데, 이는 매우 드문 일입니다. 만약 우리가 N 개의 다른 원소를 해싱했을 때, R_{max} 가 k 라면, 이는 2^k 정도의 원소를 시도했을 때 한 번 나올 법한 희귀한 패턴이 나타났다는 것을 의미합니다. 따라서 고유 원소의 개수를 2^k (여기서는 $2^{R_{max}}$)라고 추정하는 것입니다.

문제 풀이:

주어진 정보:

문제에서는 "가장 오른쪽 1이 처음 등장하는 위치"를 기반으로 추정하라고 되어 있습니다. 이는 $R(x)$ 를 계산하는 것과 같습니다. (위치의 시작을 0으로 가정합니다.)

값	해시값 (8bit)	가장 오른쪽 1의 위치 (R)
A	01010000	4
B	00010010	1
C	00100000	5
D	10000001	0

1. 각 해시값에 대해 R값 계산:

- **A (01010000)**: 오른쪽에서 5번째 비트가 처음 나오는 1입니다. (위치를 0부터 세면 $R=4$)
- **B (00010010)**: 오른쪽에서 2번째 비트가 처음 나오는 1입니다. ($R=1$)
- **C (00100000)**: 오른쪽에서 6번째 비트가 처음 나오는 1입니다. ($R=5$)
- **D (10000001)**: 오른쪽에서 1번째 비트가 처음 나오는 1입니다. ($R=0$)

2. R의 최댓값($R_{\max}$) 찾기:

계산된 R 값들 {4, 1, 5, 0} 중에서 최댓값은 5입니다.

- $R_{\max} = 5$

3. 고유 원소 수 추정:

고유 원소의 수는 $2^{R_{\max}}$ 로 추정합니다.

- 추정된 고유 원소 수 = $2^5 = 32$

답:

Flajolet-Martin 알고리즘에 따라 추정된 고유 원소의 수는 32입니다.

참고: 실제 Flajolet-Martin 알고리즘은 정확도를 높이기 위해 여러 개의 해시 함수를 사용하고 그 결과의 평균(또는 중앙값)을 사용하는 등 더 복잡한 과정을 거치지만, 기본 원리는 위와 같습니다.

㉠ **Q13. Spearman Correlation** 계산 결과, 국어 성적과 영어 성적 간의 순위 상관계수 $\rho = 0.92$ 가 도출되었다. 이 결과의 의미를 서술하시오.

㉡ 상세 풀이 >

스피어만 순위 상관계수(Spearman Correlation)의 의미

상관계수는 두 변수 간의 관계가 얼마나 강한지를 나타내는 -1에서 1 사이의 값입니다. 스피어만 순위 상관계수(ρ 또는 r_s)는 일반적인 피어슨 상관계수(Pearson Correlation)와 중요한 차이점이 있습니다.

- 피어슨 상관계수: 두 변수 간의 선형(linear) 관계를 측정합니다. 변수의 실제 값(value)을 사용하여 계산합니다.
- 스피어만 순위 상관계수: 두 변수 간의 단조(monotonic) 관계를 측정합니다. 변수의 실제 값 대신, 값의 순위(rank)를 사용하여 계산합니다.

단조 관계(Monotonic Relationship)란?

한 변수가 증가할 때 다른 변수도 항상 같이 증가하거나(단조 증가), 항상 같이 감소하는(단조 감소) 관계를 의미합니다. 이 관계가 반드시 직선일 필요는 없습니다. 아래와 같은 곡선 관계도 단조 관계에 포함됩니다.

문제 풀이:

주어진 정보:

"국어 성적과 영어 성적 간의 순위 상관계수 $\rho = 0.92$ 가 도출되었다."

이 결과의 의미:

1. 매우 강한 양의 상관관계:

- 상관계수 값 0.92는 1에 매우 가깝습니다. 이는 두 변수 사이에 매우 강한 양의 관계가 존재함을 의미합니다.
- "양의 관계"란, 한 변수의 순위가 높아지면 다른 변수의 순위도 높아지는 경향이 있다는 뜻입니다.

2. 순위 기반의 관계:

- 이것은 "국어 점수"와 "영어 점수"의 관계가 아니라, "국어 등수"와 "영어 등수"의 관계를 의미합니다.
- 즉, 국어 성적 등수가 높은 학생은 영어 성적 등수도 높을 경향이 매우 강하다는 것을 뜻합니다. 반대로 국어 등수가 낮은 학생은 영어 등수도 낮을 가능성이 큼니다.

3. 선형 관계가 아닐 수 있음:

- 두 학생이 국어 점수에서 10점 차이가 날 때, 영어 점수에서도 항상 비슷한 점수 차이를 보인다는 의미는 아닙니다.
- 예를 들어, 국어 1등이 100점, 2등이 80점이고, 영어 1등이 100점, 2등이 99점일 수 있습니다. 점수 차이는 다르지만 '순위'는 1등, 2등으로 동일하게 유지됩니다. 스피어만 상관계수는 바로 이 순위의 일치성을 측정합니다.

결론:

국어 성적과 영어 성적 간의 스피어만 순위 상관계수가 0.92라는 것은, 두 과목의 상대적인 순위가 매우 유사하다는 것을 의미합니다. 국어를 잘하는 학생이 영어도 잘하고, 국어를 못하는 학생이 영어도 못하는 경향이 매우 뚜렷하게 나타난다고 해석할 수 있습니다.